
Fin alternative du projet de session

C64 | Objets connectés

Table des matières

Introduction	3
La trottinette	3
Le contrôleur.....	5
Diagramme d'états-transitions principal	5
Éléments à compléter	6
Détails sur certains contrôles et états	6
Contrôle du phare avant	6
Contrôle des clignotants gauche et droit.....	6
Contrôle du phare arrière	6
Contrôle des bandes latérales.....	6
Comportement de l'état charging	6
Comportement de l'état scooting	7
États porteurs d'une machine à états	7
États des leds pour les différents états	8
Simulation	9
Développement logiciel.....	9
MVC	9
MVCH	9
Implémentation.....	10
Acquisition à partir de la console	11
Affichage sur la console	12
Modélisation physique	13
Ajouts personnels	15
Ajout fonctionnel	15
Ajout d'un outil de développement.....	15
Rapport	16
Évaluation.....	17
Annexe – Classe console	17

Introduction

Par manque de temps, on retire du mandat original les phases 5 et 6 initialement prévues.

On les remplace par un nouveau projet réalisant le développement du contrôleur d'une trottinette électrique.

Cet énoncé est court et vise à décrire les éléments essentiels du projet. N'hésitez pas à poser des questions si des éléments du mandat doivent être clarifiés.

La trottinette

Vous devez réaliser le développement du contrôleur d'une trottinette électrique moderne. L'image suivante présente approximativement le prototype en cours de développement.



Représentation artistique produite par Gemini

La trottinette possède :

- Diodes électroluminescentes (del) :
 - Un phare avant blanc servant à l'éclairage.
 - Un phare arrière rouge indiquant le freinage.
 - Quatre clignotants jaunes se trouvant à gauche et à droite :
 - à l'avant, on les retrouve à l'extrémité des poignées;
 - à l'arrière, ils sont situés immédiatement de chaque côté du feu de freinage par des bandes jaunes visibles autant de l'arrière que de chaque côté.
 - Des bandes décoratives bleues se trouvant de chaque côté au niveau du marchepied.
- Contrôle de la trottinette :
 - Un bouton servant à la mise en route ou à la fermeture du système électrique.
 - Un bouton d'activation/désactivation du phare avant.
 - Deux boutons d'activation/désactivation des clignotants gauche et droit.
 - Une gâchette au pouce permet de régler l'accélération.
 - Des manettes situées devant les poignées permettent le freinage.
- Console de rétroaction : quelques voyants servent à donner de l'information à l'utilisateur :
 - Au-dessus, deux voyants multicolores :
 - à gauche, indiquant l'état général de la trottinette;
 - à droite, partiellement utilisé, pourra servir à l'ajout personnel si nécessaire.
 - Trois voyants à barres indiquant respectivement les niveaux associés à :
 - la vitesse de la trottinette;
 - le niveau de charge de la batterie;
 - la température de la batterie.
- Une prise de branchement pour le câble de recharge de la batterie est disponible. À l'interne, un capteur permet de détecter si un câble est présent ou non.

Le contrôleur

Le contrôleur de la trottinette est la composante logicielle qui tourne en boucle fermée en tout temps. Ce contrôleur est le seul logiciel en exécution constante à l'exception du système d'exploitation. Ce logiciel utilise une couche logicielle interfaçant les entrées et sorties électroniques et électriques. Cette couche logicielle est une bibliothèque similaire à **EasyGoPiGo** qui servait à contrôler le robot **GoPiGo** (cette partie sera simulée).

Diagramme d'états-transitions principal

Le diagramme UML d'états-transitions suivant présente une partie du contrôleur à produire

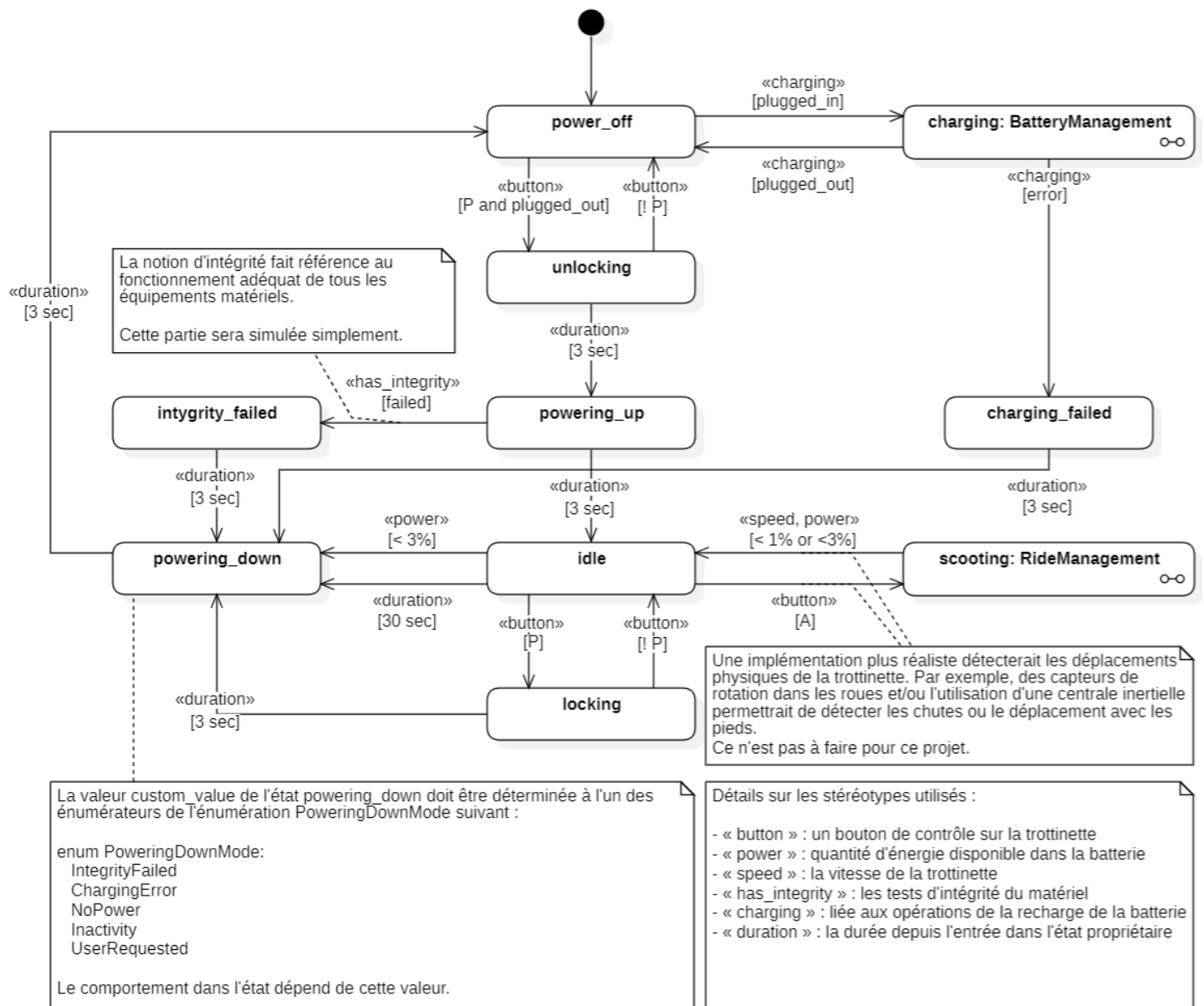


Diagramme UML d'états-transitions représentant l'application principale

Éléments à compléter

Ce diagramme est volontairement incomplet et implique de nouvelles notions de généralisation. C'est votre tâche que d'identifier les meilleures approches à utiliser. Bien qu'il existe toujours plusieurs façons de réaliser des implémentations en informatique, la qualité de votre solution ne repose pas exclusivement sur l'aspect fonctionnel, mais aussi sur la qualité de votre solution. Les éléments non présentés sont :

1. la gestion des clignotants gauche et droit
2. la gestion du phare avant
3. les comportements des sous-machines d'états et de leur gestion interne :
 - a. **BatteryManagement**
 - b. **RideManagement**
4. quelques autres détails dont certaines implications des choix faits

Détails sur certains contrôles et états

Contrôle du phare avant

Le phare peut être ouvert ou fermé en tout temps lorsque l'état courant est : **power_off**, **charging**, **idle** et **scooting**.

Contrôle des clignotants gauche et droit

Les clignotants peuvent être ouverts ou fermés en tout temps lorsque l'état courant est : **idle** et **scooting**.

Contrôle du phare arrière

Le phare arrière peut être activé ou désactivé seulement lorsque l'état courant est : **idle** et **scooting**.

Ce phare indique l'amplitude du freinage appliqué par l'utilisateur sur les manettes de frein. Autrement dit, une gradation de l'intensité lumineuse reflète l'amplitude du freinage.

Sachez aussi que le phare arrière sert de rétroaction visuelle dans d'autres situations. Ces comportements automatiques sont destinés à informer l'utilisateur d'un problème et ne sont utilisés que lorsque l'utilisateur ne peut les manipuler par les manettes de frein.

Contrôle des bandes latérales

Les bandes latérales sont principalement décoratives, mais donnent parfois une rétroaction sur l'état en cours. Leur contrôle est entièrement automatisé et décrit plus bas.

Comportement de l'état **charging**

Le chargement de la batterie n'est possible qu'avec les conditions suivantes :

- la trottinette est en veille (l'état courant est **power_off**)
- la détection du câble de recharge active l'état **charging**
- le retrait du câble de recharge revient à l'état **power_off**.

Lorsque l'état courant passe à **charging** :

- Automatiquement, le chargement est démarré.
- Pendant la charge, si la température de la batterie est strictement supérieure à 85°C, on passe en mode de refroidissement. Le refroidissement est passif et seul l'attente peut refroidir la batterie. La recharge reprend automatiquement lorsque la température est strictement inférieure à 75°C.

- Pendant la charge, si le niveau de charge est supérieur ou égal à 99%, on considère la charge complète. Dans ce cas, on cesse le chargement. On reprendra le chargement seulement lorsque le niveau de charge sera strictement inférieur à 95%.
- Pendant la charge, il peut survenir une erreur technique quelconque détectée par les capteurs. Si cette situation se présente, on termine entièrement le comportement de chargement de la batterie.
- Le phare avant peut être allumé ou fermé sans restriction pendant le chargement.

Comportement de l'état **scooting**

Lorsque l'utilisateur est à l'arrêt (état **idle**) et qu'il active l'accélération, il entre en mode de gestion des déplacements.

Les considérations à respecter sont :

- Automatiquement, on entre en mode *roue libre* où l'utilisateur peut :
 - se laisser glisser
 - utiliser ses pieds pour contrôler la trottinette (aucune gestion explicite pour cette partie)
 - utiliser les contrôles de la trottinette pour accélérer ou freiner.
- Pendant le mode *roue libre* : s'il appuie sur l'accélérateur :
 - il accélère selon la consigne donnée (un pourcentage de la puissance maximum du moteur). S'il relâche l'accélérateur, il revient en mode *roue libre*. Lorsqu'il accélère, il consomme de l'énergie et chauffe la batterie.
- Dans tous les cas, s'il appuie sur l'un des freins :
 - il freine selon la consigne de freinage (un pourcentage de la capacité totale de freinage)
 - si l'accélérateur est actif, l'accélération est ignorée
 - si tous les freins sont relâchés, il revient en mode *roue libre*
 - pendant le freinage, de l'énergie est restaurée, mais chauffe la batterie
- Le phare avant et les clignotants peuvent être allumés ou fermés sans restriction.

Cette version du projet ne gère pas les cas où l'utilisateur n'utilise pas le moteur électrique pour déplacer la trottinette. Autrement dit, on omet la partie où il utilise ses pieds pour accélérer.

États porteurs d'une machine à états

Attention, deux états sont des états ayant des sous-processus. C'est-à-dire que ces états possèdent chacun une machine à états en leur sein.

On reconnaît ces états grâce à ce petit symbole se trouvant en bas à droite de l'état : . Ce symbole représente deux états reliés par une transition (deux petits cercles reliés par une ligne).

Il est impératif de comprendre que ces états ne sont pas des **TrackingDevice** et que les machines à états contenues dans des états ne sont pas des **sub-devices**. Il est attendu que vous fassiez une interprétation et une implémentation de haut niveau. Puisque c'est un élément récurrent, il faut se poser la question si le concept est générique et réutilisable. Si oui, quelles sont les contraintes et structures à valoriser afin de maximiser le DRY et l'effort d'abstraction initial.

États des leds pour les différents états

	Phare avant	Clignotant gauche	Clignotant droit	Phare arrière de freinage	Bandes latérales	Indicateur d'état gén.	Indicateur personnalisé	Indicateur vitesse	Indicateur charge	Indicateur température
Type de liaison	-	avant arrière	avant arrière	-	gauche droite	-	-	-	-	-
power_off										
charging		-	-	-	 (2.0)[75%]{b}	 (2.0)[75%]	 (2.0)[5%]	-	 (2.0)[75%]	 (2.0)[75%]
unlocking	-	-	-	-	 (0.5)[50%]{r}	 (0.5)[75%]{b}	 (0.5)[75%]{b}	-	-	-
powering_up	-	-	-	-	 (0.75)[75%]{b}	 (0.75)[75%]{b}	 (0.75)[75%]{b}	-	-	-
idle		 (1.0)[50%]{r}	 (1.0)[50%]{r}		 (2.0)[75%]{b}		-			
scooting		 (1.0)[50%]{b}	 (1.0)[50%]{b}		 (2.5)[90%]{b}	 (2.5)[90%]	-			
powering_down					 (0.5)[25%]{b}	 (0.5)[50%]{r}	Avec ces couleurs selon le contexte : - IntegrityFailed : - ChargingError : - NoPower : - Inactivity : - UserRequested :			
locking	-	-	-	-	 (0.5)[50%]{r}	 (0.5)[75%]{b}	 (0.5)[75%]{b}	-	-	-
integrity_failed	-	-	-	 (0.5)[25%]		 (0.5)[25%]	-	-	-	-
charging_failed	-	-	-	 (0.5)[25%]		 (0.5)[75%]	-	-	-	-

Le tableau suivant constitue la légende des diagrammes utilisés dans le tableau précédent.

Titre	Représentation	Description
Aucune action	-	Aucun changement appliqué.
Éteint		Indicateur complètement éteint.
Couleur solide vive		Ouvert pleine puissance de la couleur indiquée.
Couleur solide faible		Ouvert faible puissance de la couleur indiquée.
Activation manuelle		Activation et désactivation manuelle par l'utilisateur selon les couleurs données.
Clignotement automatique pour un del	 (d)[p]	Clignotement automatique avec les couleurs données, pendant la durée d et la proportion allumée p .
Clignotement automatique pour deux dels liés	 (d)[p]{m}	Clignotement automatique avec les couleurs données, pendant la durée d , la proportion allumée p et le mode m ($b = both$ et $r = reciprocal$).
Indicateur de niveau		Indicateur selon le niveau indiqué.
Clignotement de l'indicateur de niveau	 (d)[p]	Clignotement automatique de l'indicateur avec les couleurs données et de la durée d et de la proportion allumée p .

Simulation

Le prototype électromécanique de la trottinette n'étant pas disponible, on vous demande de simuler le contrôleur à partir de la console, autant en entrée qu'en sortie.

Par conséquent, vous devez simuler :

- L'affichage des DEL par des graphiques dans la console.
- L'activation des boutons et du câble électrique par des touches du clavier.

Développement logiciel

Vous devez produire une implémentation rigoureuse du patron de conception **MVC** que vous avez vu dans quelques cours. Assurez-vous de poser des questions si vous désirez plus de détails sur le sujet.

Plusieurs approches sont possibles ici. Toutefois, on encourage l'une des deux approches suivantes : MVC direct ou MVCH, modèle-vue-contrôleur-hiérarchique.

MVC

Vous faites la fusion directe des deux projets. Autrement dit, la simulation devient le corps du projet. On oublie l'implémentation physique et on redirige tout vers la console; autant les intrants que les extrants.

Cette approche est plus facile à tous les points de vue. Néanmoins, elle a le désavantage d'être très peu réutilisable dans l'implémentation réelle du projet.

MVCH

Puisque vous faites une simulation d'un système complet, il est possible de considérer votre projet comme étant une imbrication hiérarchique de 2 MVC (MVCH). Généralement, pour ce type d'implémentation, le MVC interne constitue le modèle du MVC externe.

Le tableau suivant présente quelques considérations sur cet aspect :

	Trottinette – MVC interne	Simulation – MVC externe
Modèle	- Correspond à ce que vous auriez comme outil de développement si vous aviez à réaliser réellement ce projet, c'est-à-dire une interface de programmation vers les constituants électriques de la trottinette. - Exactement comme la bibliothèque EasyGoPiGo3 le fait pour contrôler le robot GoPiGo3.	- Correspond au MVC de la trottinette. - Autrement dit, une partie ou tous les éléments de la colonne de gauche.
Vue	Les nombreux témoins sur la trottinette ainsi que la console.	L'affichage dans la console.
Contrôleur	L'application TrackingApplication.	Un programme écoutant et redirigeant les touches du clavier.
Usager	Interagit directement avec les boutons et contrôles de la trottinette.	Interagit directement avec la console via le clavier de l'ordinateur.

Implémentation

Les diagrammes suivants présentent trois variantes possibles du MVC.

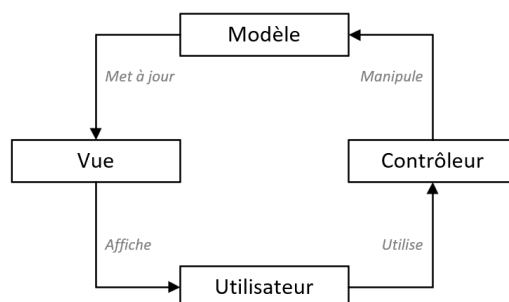


Diagramme des interactions d'une implémentation classique du MVC

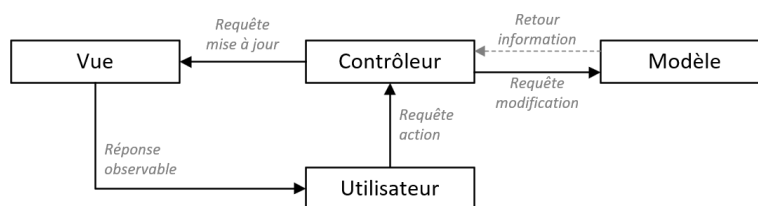


Diagramme des interactions d'une implémentation se rapprochant du MVC Model 2

Avec cette approche, la vue est passive et un patron observateur la met à jour.

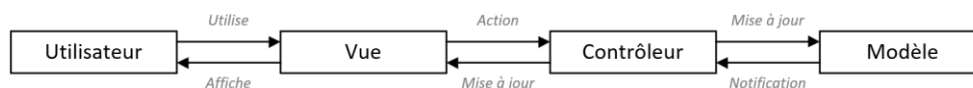


Diagramme des interactions d'une approche se rapprochant d'un MVP, une variante du MVC

Avec le modèle-vue-présentation, le contrôleur a le rôle d'intermédiaire et sert de présentateur pour redistribuer les intrants de l'utilisateur et afficher les données.

Acquisition à partir de la console

Le tableau suivant présente la correspondance entre les touches et les actions à poser :

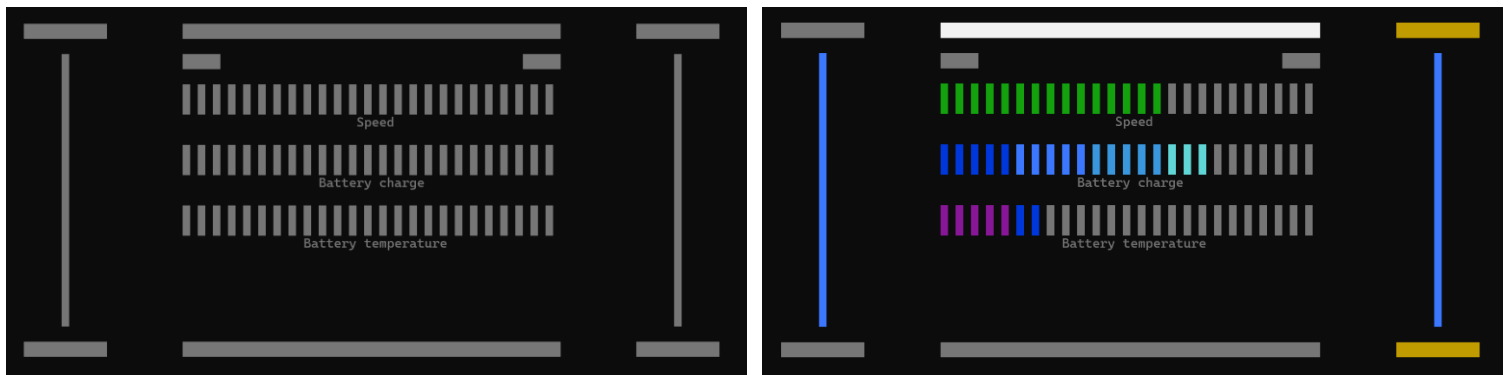
Touche	Action
Barre d'espace	Bouton de bascule du phare avant.
I et O	Action d'insertion et retrait du câble de chargement. Notes : on simule l'activation et la désactivation du capteur de détection du câble par ces touches. Autrement dit, on simule que : • en appuyant sur I, le capteur détecte la présence du câble d'alimentation jusqu'à l'appui de la touche O (attention, si la trottinette n'est pas éteinte, la touche I est sans effet) • en appuyant sur O, le capteur ne détecte plus la présence du câble d'alimentation (attention, si la trottinette n'est pas en train de charger, la touche O est sans effet)
F	Force une erreur : • de validation d'intégrité • de chargement de la batterie Cette touche simule une faute lors de la validation d'intégrité dans l'état powering_up ou de chargement de la batterie pendant sa charge. Dans tous les autres cas, la touche F est sans effet.
P	Bouton de mise en route ou de fermeture.
Flèche avant et arrière	Accélération et freinage. Notes : les contrôleurs physiques d'accélération et de freinage donnent des valeurs continues alors qu'une touche au clavier ne peut donner qu'une valeur discrète. Pour ce projet, l'accélération simulée sera d'amplitude moyenne alors que le freinage simulé sera de forte amplitude.
Flèches gauche et droite	Boutons d'activation/désactivation des clignotants gauche et droit.
Au choix	Il est possible d'ajouter d'autres comportements au choix. Toutefois, vous devez afficher au début du programme quelles nouvelles actions sont déclenchées par quelles nouvelles touches.

Affichage sur la console

La console doit présenter les 13 zones graphiques de rétroactions visuelles. Les captures d'écran suivantes représentent le visuel à produire.



Présentation des indicateurs



Tous les indicateurs sont fermés Un exemple arbitraire en opération

Les constituants visuels suivants sont interreliés pendant la simulation :

- les clignotants droits sont toujours synchronisés : avant et arrière
- les clignotants gauches sont toujours synchronisés : avant et arrière
- les deux indicateurs sont parfois synchronisés

Modélisation physique

Une modélisation physique simpliste a été développée pour ce projet. Les équations suivantes simplifient grandement la réalité, mais permettent une implémentation facile et une simulation crédible.

Les fonctions d'approximation du modèle sont :

- $E_{t+1} = \max(0, \min(E_t + \Delta t P_{net}, P_{max}))$
- $T_{t+1} = T_t + \Delta t (T_e P_c^2 - T_d (T_t - T_a))$
- $$v_{t+1} = \begin{cases} \max(0, v_t - \Delta t (C_r + K_b a_b + C_a v_t^2)) & \text{pendant roue libre ou freinage} \\ v_t + \Delta t \left(K_a a_a \left(1 - \frac{v_t}{v_v} \right) - C_a v_t^2 \right) & \text{pendant accélération} \end{cases}$$

Le tableau suivant présente comment le modèle est appliqué selon le contexte.

	Gestion de la puissance nette	Gestion de la vitesse
power_off	$P_{net} = -P_s$	-
charging	$P_{net} = \begin{cases} -(P_i + P_m a_t) & \text{en recharge} \\ -P_i & \text{en refroidissement} \\ -P_i & \text{en attente pleine charge} \end{cases}$	-
unlocking	$P_{net} = -P_i$	-
powering_up	$P_{net} = -P_i$	-
idle	$P_{net} = -P_i$	-
scooting	$P_{net} = \begin{cases} -P_i & \text{pendant roue libre} \\ -(P_i + P_m a_a) & \text{pendant accélération} \\ E_r a_b v_t - P_i & \text{pendant freinage} \end{cases}$	Voir plus bas la fonction : $v_{t+1} = \dots$
locking	$P_{net} = -P_i$	-
powering_down	$P_{net} = -P_i$	-
integrity_failed	$P_{net} = -P_i$	-
charging_failed	$P_{net} = -P_i$	-

Les tableaux suivants donnent tous les détails sur les variables et constantes utilisées.

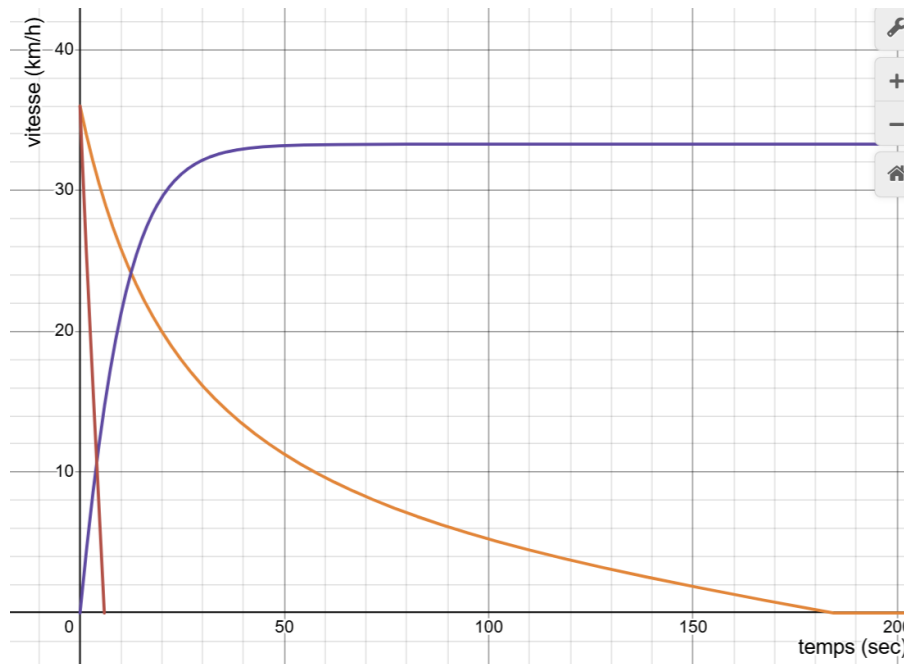
Variables principales liées aux modèles			
Symbole	Description	Valeur	Unité
v_t	vitesse au temps t	formule	$m \cdot s^{-1}$
E_t	énergie stockée au temps t	formule	$J = W \cdot s$
T_t	température de la batterie au temps t	formule	$^{\circ}C$
P_{net}	puissance nette transmise (positive = recharge, négative = décharge)	formule	W

Variables générales du temps écoulé et de contrôle par l'utilisateur			
Symbole	Description	Valeur	Unité
Δ_t	variation de temps entre deux tics	-	s
a_a	accélération due à la consigne d'accélération au temps t (en pourcentage)	0.75 valeur discrète utilisée pour la simulation	-
a_b	décélération due à la consigne de freinage au temps t (en pourcentage)	0.50 valeur discrète utilisée pour la simulation	-

Constantes reliées au système électro mécanique et à l'aérodynamisme			
Symbole	Description	Valeur	Unité
C_r	coefficient de résistance au roulement	0.0153	$m \cdot s^{-2}$
C_a	coefficient global de décélération aérodynamique massique	0.0037	m^{-1}
V_v	vitesse maximum du moteur à vide	15.000 (~54 km/h)	$m \cdot s^{-1}$
K_a	accélération maximale absolue sans charge	1.1	$m \cdot s^{-2}$
K_b	décélération maximale lorsque les freins sont appliqués au maximum	5.0	$m \cdot s^{-2}$

Constantes reliées aux systèmes électriques et à la batterie			
Symbole	Description	Valeur	Unité
E_r	constante de régénération électromécanique	120.00	$N = W \cdot m \cdot s^{-1}$
E_{max}	énergie maximum de la batterie	1.5e6	$J = W \cdot s$
P_m	puissance maximale consommée par le moteur	120.00	W
P_l	puissance consommée par l'électronique sans puissance	2.50	W
P_s	puissance consommée par l'électronique en état de veille	0.05	W
P_c	valeur absolue de la puissance traversant la batterie	$P_c = P_{net} $	W
T_e	coefficient d'échauffement interne de la batterie (inclut la résistance interne, la tension nominale et la capacité thermique massique)	5.0e-8	$^{\circ}C \cdot W^{-2} s^{-1}$
T_d	coefficient de dissipation thermique de la batterie	5.0e-4	s^{-1}

Constantes environnementales			
Symbole	Description	Valeur	Unité
T_a	température ambiante au temps t	20.0 valeur fixe pour simulation	$^{\circ}C$



Graphique de la vitesse dans le temps

En orange : vitesse en mode roue libre

En mauve : vitesse avec une accélération constante à 75% ($a_a = 0.75$)

En rouge : vitesse avec freinage à 50% ($a_b = 0.50$)

Ajouts personnels

Vous devez produire deux ajouts personnels à ce projet. On s'attend à ce que ces ajouts totalisent environ 3 heures de travail. Ce n'est pas la taille des ajouts mais la pertinence de ces derniers qui compte.

Ajout fonctionnel

On vous demande d'ajouter une fonctionnalité pertinente liée à l'utilisation de la trottinette. Cet ajout doit refléter un comportement perceptible, évident et qui serait susceptible d'augmenter l'intérêt des éventuels acheteurs.

Ajout d'un outil de développement

La bibliothèque créée depuis le début de la session correspond à une amorce intéressante d'un projet plus large, mais elle reste à bonifier de plusieurs façons.

On vous demande de proposer, justifier, concevoir et implémenter un tel ajout. L'objectif est d'ajouter un constituant au projet. On vise soit un élément :

- entièrement générique et réutilisable à travers divers projets (comme pour les phases 1 à 3)
- partiellement générique et réutilisable à travers divers projets (comme pour la phase 4)
- réutilisable spécifiquement dans le projet de la trottinette mais qui soit pertinent.

Rapport

Vous devez produire à la racine du projet un fichier `rapport-trottinette.md` et répondre aux questions suivantes :

- Équipe
 - Identifiez tous les membres de l'équipe selon ce patron : Nom, Prénom : Matricule
- États ayant une machine d'états
 - Décrivez techniquement comment vous avez créé les états possédant une machine d'états à l'interne.
 - Expliquez ce qu'apporte cette nouvelle abstraction.
 - Si vous pouviez modifier les 3 premiers niveaux de la bibliothèque, quelle serait la meilleure façon d'intégrer ce nouveau concept en le considérant générique?
- Compréhension générale
 - Le diagramme d'états-transitions initialement donné pour le contrôle de la trottinette présente au moins deux limitations importantes sur le comportement des opérations de la trottinette. Ces limitations sont volontairement présentes pour simplifier le projet et permettre la présente discussion. L'une de ces limitations a peu d'impact sur les opérations alors que l'autre présente une faute conceptuelle grave. On vous demande d'identifier et d'expliquer votre raisonnement pour ces deux éléments. Sachez que plusieurs réponses sont possibles. Votre réponse doit être courte et aucun correctif ne vous est demandé.
- Évaluation des objectifs :
 - On vous demande de produire les deux listes suivantes identifiant les éléments du mandat qui :
 - ont été partiellement réalisés (quoi et jusqu'à où)
 - n'ont pas été réalisés.
- Ajout personnel
 - Décrivez l'objectif principal de votre ajout personnel. Si vous en avez plusieurs, présentez-les dans l'ordre de pertinence.
 - Décrivez brièvement comment vous avez réalisé l'intégration technique.
- Diagrammes UML
 - Vous devez produire les diagrammes UML suivants :
 - diagramme de classe de votre vue (toutes les classes impliquées);
 - diagramme de classe de votre contrôleur (toutes les classes impliquées);
 - diagramme de classe de votre modèle (toutes les classes impliquées);
 - diagramme d'états-transitions pour la gestion du phare avant, des clignotants et des états `charging` et `scooting`.
 - Ces 4 diagrammes doivent être remis dans les 4 fichiers suivants à la racine du projet :
 - `uml_vue.png`
 - `uml_controleur.png`
 - `uml_modele.png`
 - `uml_etats_transitions.png`
- Pour chacun des étudiants de l'équipe, on vous demande de répondre aux 2 questions suivantes. Même si deux étudiants ont des réponses similaires c'est acceptable en autant que chacun l'exprime à sa façon.

- Considérant les phases 1 à 4 de la bibliothèque, nommez les deux parties que vous avez trouvé le plus intéressant et le plus difficile.
- Considérant le mandat de la fin alternative, discuter de sa pertinence dans le cadre de réalisation d'un projet valorisant la bibliothèque développées.
- Optionnellement, tout commentaire constructif sera considéré pour la mise à jour du projet et du cours.

Toutes vos réponses doivent être concises, précises et utiliser un vocabulaire ultra technique. Si vous êtes capable d'exprimer un point adéquatement avec 15 mots vous aurez 100%. Si vous utilisez 500 mots pour décrire un élément simple, vous perdrez des points.

Aucun usage de l'IA n'est accepté pour ce rapport (autant pour l'analyse, la production d'idée que pour la rédaction).

Évaluation

Les critères suivants sont en ordre d'importance d'évaluation :

1. Valorisation de la bibliothèque créée, c'est-à-dire que vous exploitez au maximum les divers concepts couverts et que vous ajoutez les éléments manquants en respectant la philosophie employée.
2. La qualité de la conception présentée par les diagrammes UML.
3. La qualité de votre implémentation et la justesse de la correspondance entre votre code et votre conception.
4. La fonctionnalité.
5. La qualité du code incluant les annotations de type.

La pondération de fin de session faite via le fichier d'auto-évaluation de l'équipe sera appliquée pour cette partie.

Annexe – Classe `console`

Le module `console` vous donne accès à la classe `Console` qui facilite toutes les interactions avec les opérations de la console pour ce projet.

La classe présente dans sa `docstring` un résumé des outils à votre disposition. Se trouve au bas du fichier plusieurs exemples d'utilisation.